

System Programming Project 3

담당 교수 : 김영재 교수님

이름 : 백서연

학번 : 20221197

1. 개발 목표

본 프로젝트의 목표는 다수의 클라이언트가 동시에 접속하여 주식 조회, 구매, 판매 명령을 처리할 수 있는 동시 주식 서버를 구현하는 것이다. Task 1에서는 select 기반 이벤트 드리븐 방식을, Task 2에서는 pthread 기반 스레드 풀 방식을 도입하고, Task 3에서는 두 방식의 성능을 정량적으로 비교·분석한다.

2. 개발 범위 및 내용

A. 개발 범위

Task 1: Event-driven Approach

- ✓ 다수의 클라이언트 요청을 select()를 이용해 비동기적으로 처리하여 주식 명령(show, buy, sell)이 정상 수행됨을 확인함.

- Task 2: Thread-based Approach

- ✓ Master/Worker 구조의 pthread 기반 서버로 클라이언트 연결을 공유 버퍼에 분배·병렬 처리하여 높은 동시 처리율을 달성함.

- Task 3: Performance Evaluation

- ✓ 두 방식의 Throughput 및 Latency 를 정량적으로 비교·분석하여 Thread pool 방식이 보다 우수한 확장성과 낮은 지연을 보임을 검증함.

B. 개발 내용

- Task1 (Event-driven Approach with select())

- ✓ 다수 클라이언트의 요청에 대해 select()를 이용해 FD 집합에서 읽기 준비된 소켓만 처리하는 I/O 멀티플렉싱을 구현함.
- ✓ select()는 매 호출마다 모든 FD 를 스캔하므로 FD 수 증가 시 비용이 증가하는 반면, epoll()은 이벤트 기반 레벨/엣지 트리거를 지원하여 대규모 FD 스케일링에 유리함.

- Task2 (Thread-based Approach with pthread)

- ✓ **Master Thread:** Accept()로 들어오는 새 연결을 sbuf_t 공유 버퍼에 삽입하며, 세마포어(slots, items)와 뮤텍스(mutex)로 동기화하여 안전하게 관리함.
- ✓ **Worker Thread Pool:** 미리 생성된 40 개 워커 스레드가 세마포어 대기 후 버퍼에서 connfd 추출하여 Rio_readlineb()로 요청을 읽고 process_command()로 명령을 병렬 처리함.

- Task3 (Performance Evaluation)

- ✓ **Metric:** Throughput(TPS), Average Latency(ms).
- ✓ **이유:** 서버의 처리량과 응답 지연은 동시성 모델의 성능을 비교하는 핵심 지표이기 때문.
- ✓ **측정 방법:** 클라이언트에서 gettimeofday()로 요청 전·후 시점을 기록 후 multiclient 를 통해 N = 1, 2, 4, 8, 16, 32 클라이언트 시나리오를 실행하여 평균값 산출.
- ✓ **예상 결과:**
 - ◆ 클라이언트 수 증가에 따라 select() 방식은 select() 호출 비용으로 TPS 가 포화되고 Latency 가 증가.
 - ◆ Thread 풀 방식은 컨텍스트 스위치 비용에도 불구하고 비교적 선형적 성능 향상과 안정적 지연 유지.

C. 개발 방법

Task1 (stockserver_select.c)

init_pool(), add_client(), check_clients() 함수를 추가/수정하여 FD 감시 및 클라이언트 관리를 구현함.

process_command()에 show/buy/sell 명령 처리 로직을 삽입함.

Task2 (stockserver_threaded.c)

sbuf_t 구조체와 sbuf_init(), sbuf_insert(), sbuf_remove() 함수를 추가하여 공유 버퍼 기반 연결 큐를 구현함.

worker() 스레드 함수 및 process_command() 파싱 로직을 구현함.

sigint_handler()를 추가하여 종료 시 save_stock()을 호출하도록 함.

Task3

클라이언트 코드(multiclient 또는 stockclient)에 gettimeofday() 타이밍 코드를 삽입함.

결과 로그를 CSV 로 변환하는 스크립트를 작성함.

3. 구현 결과

두 방식 모두 show, buy, sell, exit 명령 정상 동작 확인.

Event-driven 방식은 최대 16 개 클라이언트까지 높은 TPS 유지, 이후 포화 현상 발생.

Thread pool 방식은 최대 32 개 클라이언트에서도 비교적 선형 확장성 유지.

4. 성능 평가 결과 (Task 3)

- 강의자료 슬라이드의 내용 참고하여 작성 (측정 시점, 출력 결과 값 캡처 포함)